

FSM Transitions for SDCloud

Card Side

Assumptions :

- 1) *CRC which is default turned off for SPI is not turned on in CMD0*
- 2) *We are not really worried about SDHC and SDXC operation modes within this analysis.*
- 3) *Commands that are relevant to SPI but not within our captures currently, or can be universally applied to any state in Card Initialization Mode or Data Transfer Mode*
 - a) *CMD9 (Send CSD)*
 - b) *CMD10 (Send CID)*
 - c) *CMD16 (Set Block Length (Assumed 512 by default for SPI))*
 - d) *CMD27 (Program_CSD)*
 - e) *CMD59 (CRC_ON_OFF)*
- 4) *Please note regarding the Chip Select Line (CS)*
 - a) *CS is actually low triggered.*
 - b) *CS does not have to be enabled when the data is being read or written for all the time. If it was enabled when the read or write command was issued it will read/write the data. This applies especially to multi read and write.*

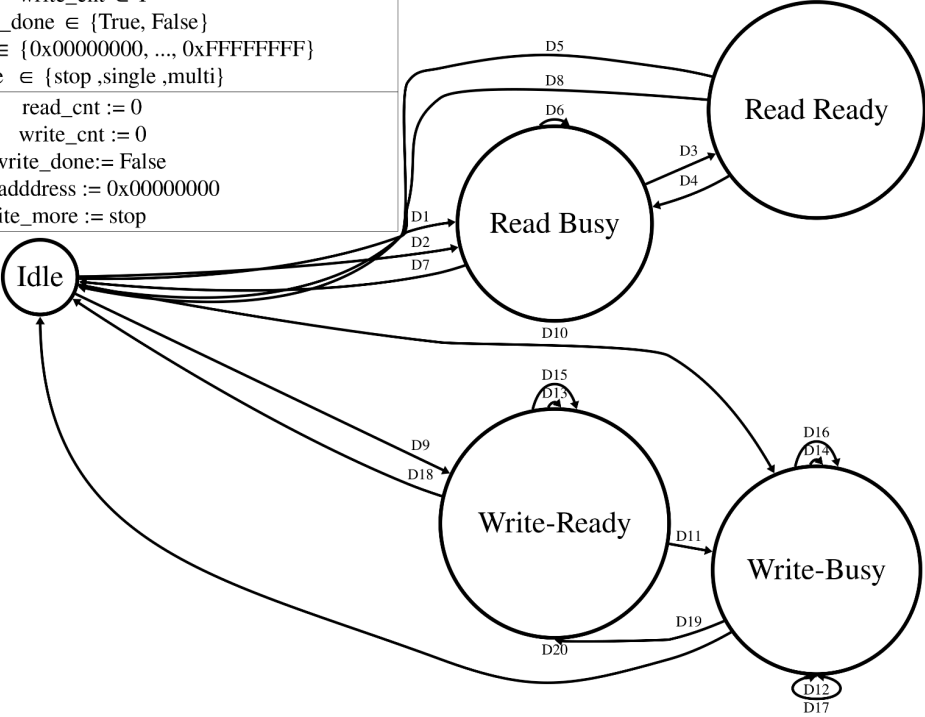
Card FSM :

Transition	Description	Trigger	Output
B1	If CS is not selected, do nothing	CS == 1	
B2	Starting the initialization process with CMD_0(GO_IDLE_STATE) and a version check	MOSI.CMD == 0 && CS == assert0 && card_ver == 2.x	MISO.RESP := 1
B3	Starting the initialization process with CMD_0(GO_IDLE_STATE) and a version check	(CS == 0) && MOSI.CMD == 0 && CS == assert1 && card_ver == 1.x	MISO.RESP := 1
B4 & B7	Typically the end of initialization of a SD card. Responds with the CCS bit in the OCR register	(CS == 0) && (MOSI.CMD == 58 && MOSI.ARG == 0 && card_ready = 1)	MISO.RESP := 1 MISO.ARG = CCS
B5	Check for the voltage carried out by a version 2.x card. If the requested voltage does not match the SD card capability. Respond with a failure	(CS == 0) && ((MOSI.CMD == 8) && (MOSI.ARG.VOLTAGE != op_voltage))	MISO.RESP := 7 ; MISO.ARG := {voltage = 0}
B6	Check for the voltage carried out by a version 2.x card. If the requested voltage matches the SD card capability. Reflect back the indicated voltage	(CS == 0) && ((MOSI.CMD == 8) && (MOSI.ARG.VOLTAGE == op_voltage))	MISO.RESP := 7 ; MISO.ARG := {voltage = op_voltage}
B8 & B11	CMD 41 initiates initialization of the SD card internal components. This command is repeated till the response has the IDLE bit set to true. Ask the Card Status	(CS == 0) && (MOSI.CMD == 41) && (card_busy == False)	ASK_CARD_STATUS
B9 & B12	If the card is ready, let the host know that the card is	CARD_FREE	MISO.RESP := 1 ; MISO.ARG := {IDLE_BIT =

	ready		0}
B10 & B13	If the card is busy initializing , let the host know that the card is not ready	CARD_BUSY	MISO.RESP := 1 ; MISO.ARG := {IDLE_BIT = 1}
B14	If command 8 is asked to a version 1.x card, it is an illegal command	(CS == 0) && MOSI.CMD == 8	MISO.RESP := 1 MISO.ARG := {Illegal_Command = 1}
B15 & B16	A command to read the OCR register of the SD card	(CS == 0) && MOSI.CMD = 58 && MOSI.ARG.REG = {OCR}	MISO.RESP := 1 DATA_OUT := REG_OCR
B17 B18 B19 B20	Host directs the card to go idle	CS == 0 && MOSI.CMD == 0	MISO.RESP := 1

Card Data Transfer Mode

States	Idle, Ready busy, Read Ready Write Ready, Write Busy
Inputs	MOSI.CMD ∈ {0, 1, ... ,59} MOSI.ARG ∈ {Data} MOSI.DATA ∈ {Data} READ_BLOCK_512(Address) ∈ {Data} CARD_BUSY ∈ {True, False} MAX_ADDRESS ∈ {0x00000000,, 0xFFFFFFFF}
Outputs	MISO.RESP ∈ {1,2,3...,7} MISO.ARG ∈ {Data} MISO.DATA ∈ {Data} WRITE_BLOCK_512(Address) ∈ {Data}
State Variables	read_cnt c I write_cnt ∈ I write_done ∈ {True, False} write_address ∈ {0x00000000, ..., 0xFFFFFFFF} write_more ∈ {stop ,single ,multi}
Initialization	read_cnt := 0 write_cnt := 0 write_done:= False write_address := 0x00000000 write_more := stop



Transition	Description	Trigger	Output	State Variables Updation
D1	Single Read command. Ask the internals to provide that specific block.	(CS == 0) && MOSI.CMD == 17	MISO.RESP := 1 BLOCK_512_ADDRESS(MOSI.ARG.ADDRESS)	Read_more := False
D2	Multiple Read	(CS == 0) &&	MISO.RESP := 1	Read_more :=

	and ask the card internals to provide the starting block of data at the mentioned address.	MOSI.CMD == 18	BLOCK_512_ADDRESS(MOSI.ARG.ADDRESS)	True read_cnt := 1 Card_busy := True
D3	The Card internals respond with a card_free when it is ready. The data is then loaded to the MISO.DATA line	CARD_FREE & read_more == True	MISO.DATA = BLOCK_512.DATA()	
D4	Continue reading the next block since we have not received a Command 12.	Read_more== True && (CS == 0) && MOSI.ARG.ADDRESS + read_cnt * 512 < MAX_ADDRESS	BLOCK_512.ADDRESS(MOSI.ARG.ADDRESS + read_cnt * 512)	Read_cnt := read_cnt + 1 Card_busy := True
D5	CMD 12 instructs the card to stop the multi-read process. The next block will not be read.	(CS == 0) && MOSI.CMD == 12	MISO.RESP := 1	Read_more := False
D6	CMD 12 instructs the card to stop the multi-read process. The next block will not be read.	(CS == 0) && MOSI.CMD == 12	MISO.RESP := 1	Read_more := False
D7	Since there is no requirement to read anymore. Return to Ready state.	CARD_FREE & read_more == False	MISO.DATA = BLOCK_512.DATA()	
D8	The address of the next block is	Read_more== True &&	MISO.RESP := 1 MISO.ARG :=	Read_cnt := 0

	illegal.	(CS == 0) && MOSI.ARG.ADDRESS + read_cnt + 512 >= Max_Address	{ILLEGAL_ADDRESS == True}	
D9	Single Write Command.	(CS == 0) && MOSI.CMD == 24	MISO.RESP := 1	Write_more := single Write_cnt := 0 Write_address := MOSI.ARG.ADDRESS
D10	Multi Write Command	(CS == 0) && MOSI.CMD == 25	MISO.RESP := 1	Write_cnt := 1 Write_address := MOSI.ARG.ADDRESS Write_more := Multi
D11	There exists some data to write and we are in the correct mode to do so. (Single / Multi)	(MOSI.DATA) && (write_more == Single write_more == multi) && (write_address + write_cnt * 512 < MAX_ADDRESS)	MISO.RESP := 1 MISO.ARG = {BUSY = True}	Write_done := False BLOCK_512{ write_address + write_cnt * 512, MOSI.DATA}
D12	The address we are supposed to write to is illegal	write_address + write_cnt * 512 > MAX_ADDRESS)	MISO.RESP := 1 MISO.ARG = {ILLEGAL_ADDRESS = True}	Write_more := False
D13	Host just asked us for our status	(CS == 0) && MOSI.CMD == 13	MISO.RESP := 1 MISO.ARG := {BUSY=True}	
D14	Host just asked us for our status	(CS == 0) && MOSI.CMD == 13	MISO.RESP := 1 MISO.ARG := {BUSY=False}	
D15 & D16	Host asked us for the count of successfully written blocks.	(CS == 0) && MOSI.CMD == 22	MISO.RESP := 1 MISO.DATA := write_cnt - 1	
D17 & D18	Host asked us to stop the multi write. It will stop	(CS == 0) && MOSI.CMD == 12	MISO.RESP := 1	write_more:= stop

	after the current block			
D19	Card is done writing and there is more to write	CARD_FREE && Write_more == multi		
D20	Card is done writing and there is no more data to write	CARD_FREE && Write_more == stop write_more = single		Write_cnt := 0 Write_more := stop

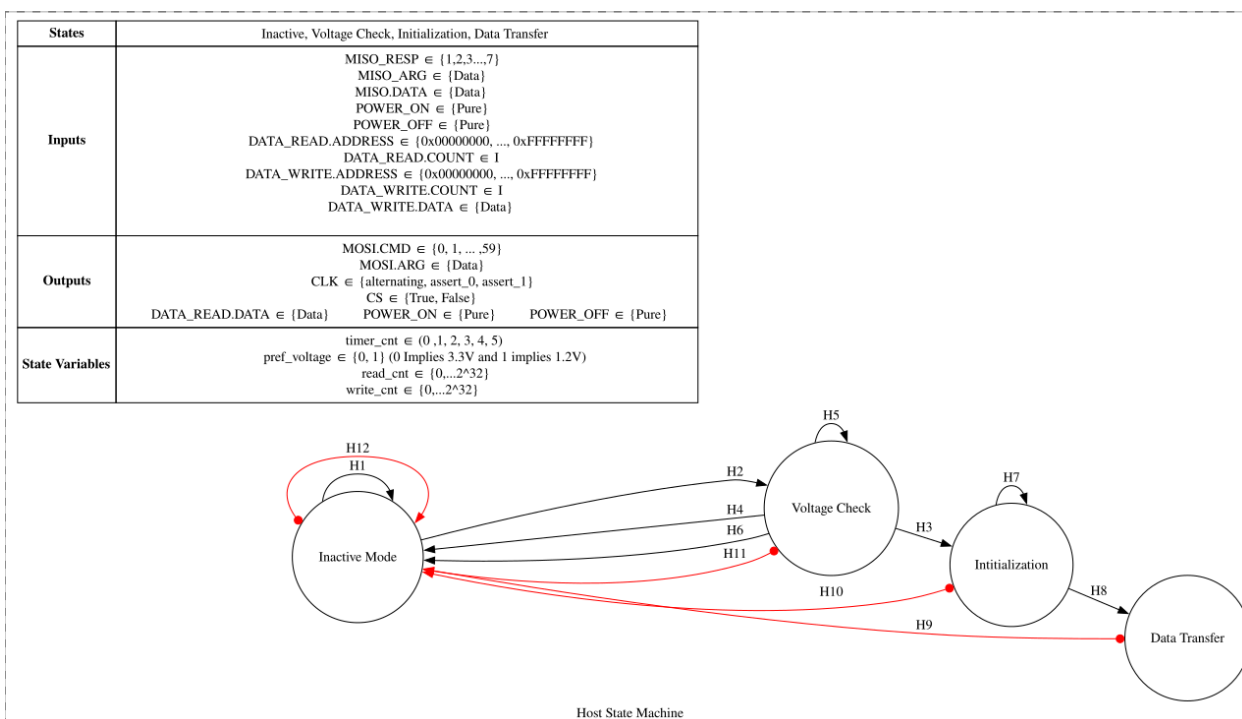
Host Side

Assumptions :

1. We are interacting with a Ver 2.x Card
2. Everytime a ACMD41 is sent there is a preceding CMD55. But since CMD55 is just needed to notify that the next command is an application command (in this case 41, we just bundle them together in ACMD41 for this FSM)

Host FSM

Please note that the timer_cnt needed in Transitions H1, H2 and H5 actually was outside the scope of our analysis as we did not really get access to a non SPI supporting SD card to calculate this precise time. We have hence added a dummy value of 5 time units for making the FSM work.



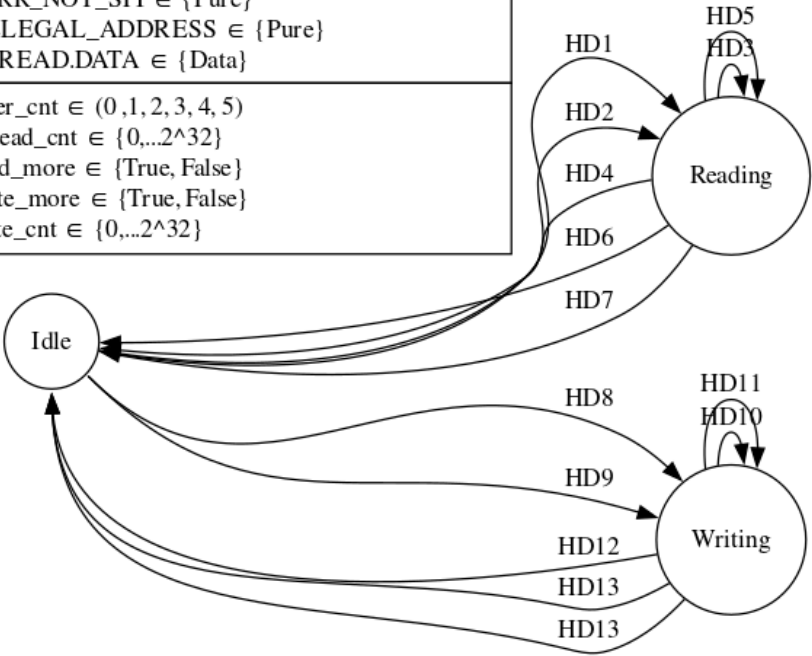
Transition	Description	Trigger	Output	State Variables Updation
------------	-------------	---------	--------	--------------------------

H1	When power is provided, Instruct the card to go to Idle state so initialization can proceed from a known state. CS == 0 is necessary for SPI mode.	POWER_ON	POWER_ON MOSI.CMD = 0 CS := 0	Timer_cnt := 5
H2	"Go Idle State" command was acknowledged. Try to figure out the operating voltage of the card.	MISO.RESP == 1	MOSI.CMD := 8 MOSI.ARG := {Operating_Voltage = preferred_voltage} CS := 0 CLK := Alternating	Timer_cnt := 0
H3	Card is using the same voltage as us. We can proceed with initializing by sending Command 41.	MISO.RESP == 7 && MISO.ARG.OPERATING_VOLTAGE == pref_voltage	MOSI.CMD := 41 CS := 0	
H4	Voltage does not match. We send an error code to the Sensor application	MISO.RESP == 7 && MISO.ARG.OPERATING_VOLTAGE != pref_voltage && MISO.ARG.OPERATING_VOLTAGE == 0	ERR_VOLTAGE	
H5	We are waiting for the response 1	Timer_cnt > 0		Timer_cnt := timer_cnt - 1
H6	We can assume that the card does not support SPI mode.	Timer_cnt == 0	ERR_NO_SPI	
H7	Keep sending ACMD41 initialization command till the card says it is ready.	MISO.RESP == 1 && MISO.ARG.IDLE_BIT == 1	MOSI.CMD := 41 CS := 0	
H8	The card is ready. We can get the Card capacity and move to	MISO.RESP == 1 && MISO.ARG.IDLE	MOSI.CMD := 58 MOSI.ARG.REG	

	Data transfer mode.	_BIT == 0	:= CCS CS := 0	
H9 H10 H11 H12	We have received Power off. Must return to the inactive mode.	POWER_OFF	POWER_OFF	Timer_cnt := 0

Host Data Transfer Mode :

States	Idle, Read Ready, Read Busy, Writing
Inputs	MISO_RESP ∈ {1,2,3...,7} MISO_ARG ∈ {Data} MISO.DATA ∈ {Data} DATA_READ.ADDRESS ∈ {0x00000000, ..., 0xFFFFFFFF} DATA_READ.COUNT ∈ I DATA_WRITE.ADDRESS ∈ {0x00000000, ..., 0xFFFFFFFF} DATA_WRITE.COUNT ∈ I DATA_WRITE.DATA ∈ {Data}
Outputs	MOSI.CMD ∈ {0, 1, ... ,59} MOSI.ARG ∈ {Data} CS ∈ {True, False} ERR_VOLTAGE ∈ {Pure} ERR_NOT_SPI ∈ {Pure} ERR_ILLEGAL_ADDRESS ∈ {Pure} DATA_READ.DATA ∈ {Data}
State Variables	timer_cnt ∈ (0,1,2,3,4,5) read_cnt ∈ {0,...,2^32} read_more ∈ {True, False} write_more ∈ {True, False} write_cnt ∈ {0,...,2^32}



Host Data Transfer State Machine

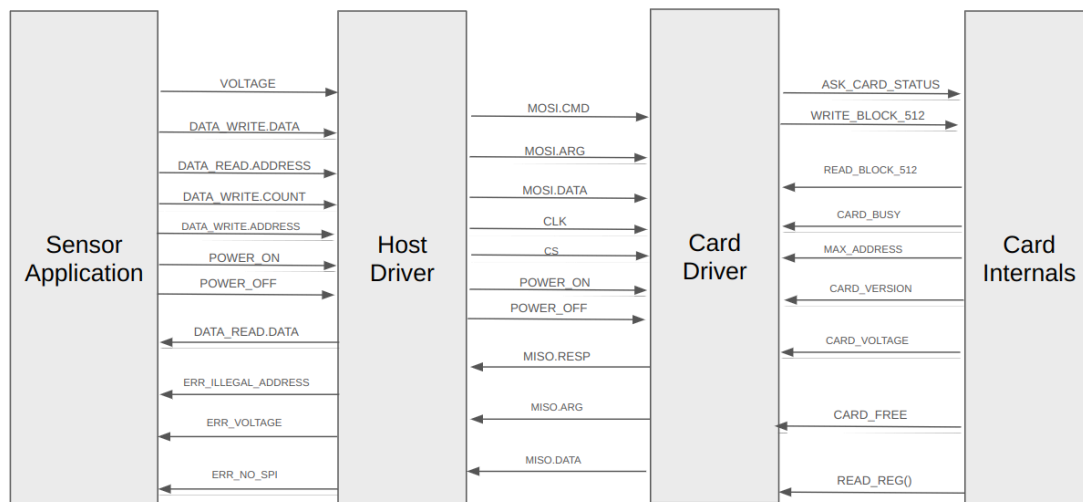
Transit ion	Description	Trigger	Output	State Variables
HD1	The Sensor needs to read multiple blocks.	DATA_READ.COUNT > 1	MOSI.CMD := 18 MOSI.ARG.ADDRESS := DATA.READ.ADDRESS CS := 0	Read_cnt := DATA_READ.COUNT - 1 Read_more := True

HD2	The Sensor needs to read a single block	DATA_READ.COUNT == 1	MOSI.CMD := 17 MOSI.ARG.ADDRESS := DATA.READ.ADDRESS CS := 0	Read_more := False
HD3	Pass the read data to the sensor application and read more data.	MISO.RESP == 1 && MISO.ARG.ILLEGAL_ADDRESS != True && read_more == True && read_cnt > 1	DATA_READ.DATA := MISO.DATA CS := 0	Read_cnt := read_cnt - 1
HD4	Pass the read data to the sensor application and stop reading data.	MISO.RESP == 1 && MISO.ARG.ILLEGAL_ADDRESS != True && read_cnt == 0	DATA_READ.DATA := MISO.DATA CS := 0	
HD5	Pass the read data to the sensor application and end reading from the next block	MISO.RESP == 1 && MISO.ARG.ILLEGAL_ADDRESS != True && read_cnt == 1	DATA_READ.DATA := MISO.DATA CS := 0 ; MOSI.CMD := 12 CS := 0	
HD6	Pass the read data to the sensor application and end single the single read operation	MISO.RESP == 1 && MISO.ARG.ILLEGAL_ADDRESS != True && read_more == False	DATA_READ.DATA := MISO.DATA CS := 0	
HD7	Tried to read an illegal address. Report error to the sensor application.	MISO.RESP == 1 && MISO.ARG.ILLEGAL_ADDRESS == True	ERR_ILLEGAL_ADDRESS	
HD8	Multi - Write initiated by the Sensor Application	DATA_WRITE.COUNT > 1	MOSI.CMD := 25 MOSI.ARG.ADDRESS := DATA.WRITE.ADDRESS; MOSI.DATA := DATA_WRITE.DATA	write_cnt := DATA_WRITE. COUNT - 1 Write_more := True

			CS := 0	
HD9	Single - Write initiated by the Sensor Application	DATA_READ.COUNT == 1	MOSI.CMD := 24 MOSI.ARG.ADDRESS := DATA.WRITE.ADDRESS; MOSI.DATA := DATA_WRITE.DATA CS := 0	write_cnt:= 0 Write_more := False
HD10	Check if the write is done.	MISO.RESP == 1 && MISO.ARG.ILLEGAL_ADDRESS != True && MOSI.ARG.BUSY == True	MOSI.CMD := 13	
HD11	If the write is done and there is more to write, write the next block to the data line.	MISO.RESP == 1 && MISO.ARG.ILLEGAL_ADDRESS != True && write_cnt != 0	MOSI.DATA := DATA_WRITE.DATA CS := 0	write_cnt := write_cnt - 1
HD12	If the single-write has completed, go back to idle state.	MISO.RESP == 1 && MISO.ARG.ILLEGAL_ADDRESS != True && write_cnt == 0 && write_more == False		
HD13	If the multi-write counter has exhausted. Stop the multi-write and go back to idle state.	MISO.RESP == 1 && MISO.ARG.ILLEGAL_ADDRESS != True && write_cnt == 0 && write_more == True	MOSI.CMD := 12 CS := 0	
HD14	Alert the Sensor application that we are trying to write to an illegal address	MISO.RESP == 1 && MISO.ARG.ILLEGAL_ADDRESS == True	ERR_ILLEGAL_ADDRESS	

Overall FSM Composition :

Asynchronous Side-by-Side Composition Of the Components



Interface	Input/Output	Description
Sensor Application <=> Host Driver	Voltage	Preferred voltage for the driver to work on {3.3V(default or 1.2 V)}
	DATA_WRITE.DATA	The Data to be written as multiples of 512 Bytes
	DATA_WRITE.ADDRESS	The starting address of the Data Blocks
	DATA_WRITE.COUNT	The Number of 512 bytes blocks
	POWER_ON	Simplified Pure Signal when the power is turned on
	POWER_OFF	Simplified Pure Signal when power is turned off (Simpler to model than have a check for power at every transition)

	ERR_VOLTAGE	The Card does not support the voltage indicated.
	ERR_NO_SPI	The SD Card does not support communication over SPI
	DATA_READ.ADDRESS	Address from which blocks of data need to be read
	DATA_READ.COUNT	Number of blocks to be read
	DATA_READ.DATA	Data read from Card
Host Driver <--> Card Driver	POWER_ON	Simplified Pure Signal when the power is turned on
	POWER_OFF	Simplified Pure Signal when power is turned off (Simpler to model than have a check for power at every transition)
	MOSI.CMD	The SD protocol command that the Card Driver initiates. Can be initialization, Reading /writing status registers and data. Usually an integer between (0 - 60). [1]
	MOSI.ARG	The arguments required to execute the command
	MOSI. DATA	The Data related to the command. (For Eg. the Data to be written to an address on the SD Card mentioned in the MOSI.ARG)
	CLK	The Clock Signal required by the SPI protocol. (usually always alternating. Not required for some specific scenarios. Like when the Card is busy reading and writing some data)
	CS	Chip Select Line required for SPI. (is Low-triggered). Used to indicate that the
	MISO.RESP	The Response generated by

		the Card Driver to the commands sent by Host Driver. Usually an integer between 1-7 for different circumstances. [2]
	MISO.ARG	The Response arguments needed for interpreting the response
	MISO.DATA	The DATA sent alongside the arguments in the response
Card Driver <=> Card Internals	WRITE_BLOCK_512.ADDRESS	The address and a block of data to which a write transaction needs to be executed.
	READ_BLOCK_512.ADDRESS	The Address from which a block of 512 bytes need to be read
	READ_BLOCK_512.DATA	The Data read from that address
	CARD_FREE	A pure signal that indicates when the card is internally ready to accept new requests (initialization and Data transfer modes) [3]
	CARD_BUSY	A pure signal that indicates when the card is internally busy and cannot service (initialization and Data transfer modes)[3]
	ASK_CARD_STATUS	Request for the card status from the Card Internals [3]

[1] (A description of all of them can be found on pg 297 of Physical Layer Simplified Specification_ Version 9.10)

[2] (Also can be found defined in the SDIO and Physical Layer Simplified Specification)

[3] Please note that this accessing of status of the card is actually trivial and almost

instantaneous (It requires checking the value stored in one of the Card Status Registers). However it has been abstracted away to stay consistent with the model of separation between the card driver and card internals.